

AI SERVICE BACKEND DEVELOPER · PORTFOLIO

틀렸는지 알 수 없는 답을 만들지 않습니다.

AI 결과를 그대로 내보내지 않고, 실패 조건을 먼저 정의한 뒤
서비스 기준 안에서 검증해 백엔드 흐름으로 연결합니다.

전지원 Jeon Jiwon

020799@naver.com · github.com/JW020799 · jw020799.github.io



전지원

AI 서비스 백엔드 개발자

020799@naver.com

github.com/JW020799

jw020799.github.io

일하는 기준 세 가지

1. 좋은 결과보다 실패 조건을 먼저 정의합니다

근거 문서가 부족한 카테고리는 FAILED로 분리하고, 재무 데이터 수집이 실패하면 unavailable로 반환합니다. 틀린 답보다 위험한 것은 틀렸는지 알 수 없는 답입니다.

2. 검증 근거는 숨기지 않고 화면에 드러냅니다

유사도 점수와 참조 문서를 응답에 그대로 노출해 사용자가 직접 판단할 수 있게 합니다.

3. 의견 차이는 같은 기준의 측정으로 좁힙니다

정확도 하나로 갈리던 팀 의견을 "오답 통과율"이라는 지표로 바꿔 측정하고 합의했습니다.

Tech Stack

Backend

- Python
- FastAPI
- SQLAlchemy
- Pydantic
- MySQL
- JWT

AI · RAG

- LangChain
- LangGraph
- ChromaDB
- Sentence-Transformers
- Ko-SRoBERTa
- Hugging Face
- OpenAI API

Infra · Frontend

- Docker
- GitHub Actions
- AWS EC2 · ECS
- React
- Vite

PROJECTS

프로젝트 3건

모두 FastAPI 백엔드 위에 AI 기능(RAG · 임베딩 검색 · Agent)을 서비스 기준으로 검증해 연결한 경험입니다.

01 · 팀 프로젝트 · 4명

TUKTAK

AI 집수리 건적 · 리스크 리포트 · 시공사 역경매 매칭

- 2026.06 ~ 2026.08
- AI 백엔드 · RAG · 이미지 유사도 검색 담당
- 오답 통과 0개 threshold, 구조화 정확도 86.4%

02 · 개인 프로젝트 · 1명

Investment Agent Report

RAG + LangGraph Agent 기반 투자 리포트 데모

- 2026.08, 수료 다음 날부터 6일
- 전체 설계 · 구현
- Agent 6개 · API 4개, 외부 API 실패 시 리포트 유지

03 · 팀 프로젝트 · 5명

GIUT

공공데이터 기반 지역 행사 탐색 · AI 추천 코스

- 2026.04 ~ 2026.05
- 백엔드 모델 · API 담당
- Event 모델 컬럼 19개 → 10개 재설계

TUKTAK

AI 기반 집수리 견적 · 리스크 리포트 · 시공사 역경매 매칭 플랫폼

사용자가 수리 사진과 설명을 올리면 AI가 기준 견적과 리스크 리포트를 먼저 만들고, 시공사 제안 비교로 이어집니다. 집수리에서 소비자가 겪는 정보 비대칭을 줄이는 것이 목표입니다.

기간	2026.06.12 ~ 2026.08.04 (8주)
인원	4명 · IBM x Red Hat AI 부트캠프 최종 프로젝트
역할	AI 백엔드 · RAG · 이미지 유사도 검색, Risk Report 도메인(Main Backend)

주요 기능

- 사진 + 설명 → 수리 요청 구조화 → AI 견적서 생성
- 근거 문서 71개 기반 RAG 리스크 리포트 (추가 비용 · 안전 · 계약 · 현장)
- 이미지 유사사례 검색으로 기존 수리 사례 매칭
- 시공사 역경매 매칭 · 제안 비교 · 채팅 · 리뷰

기술 스택

FastAPI

SQLAlchemy

MySQL

LangChain

LangGraph

ChromaDB

Hugging Face

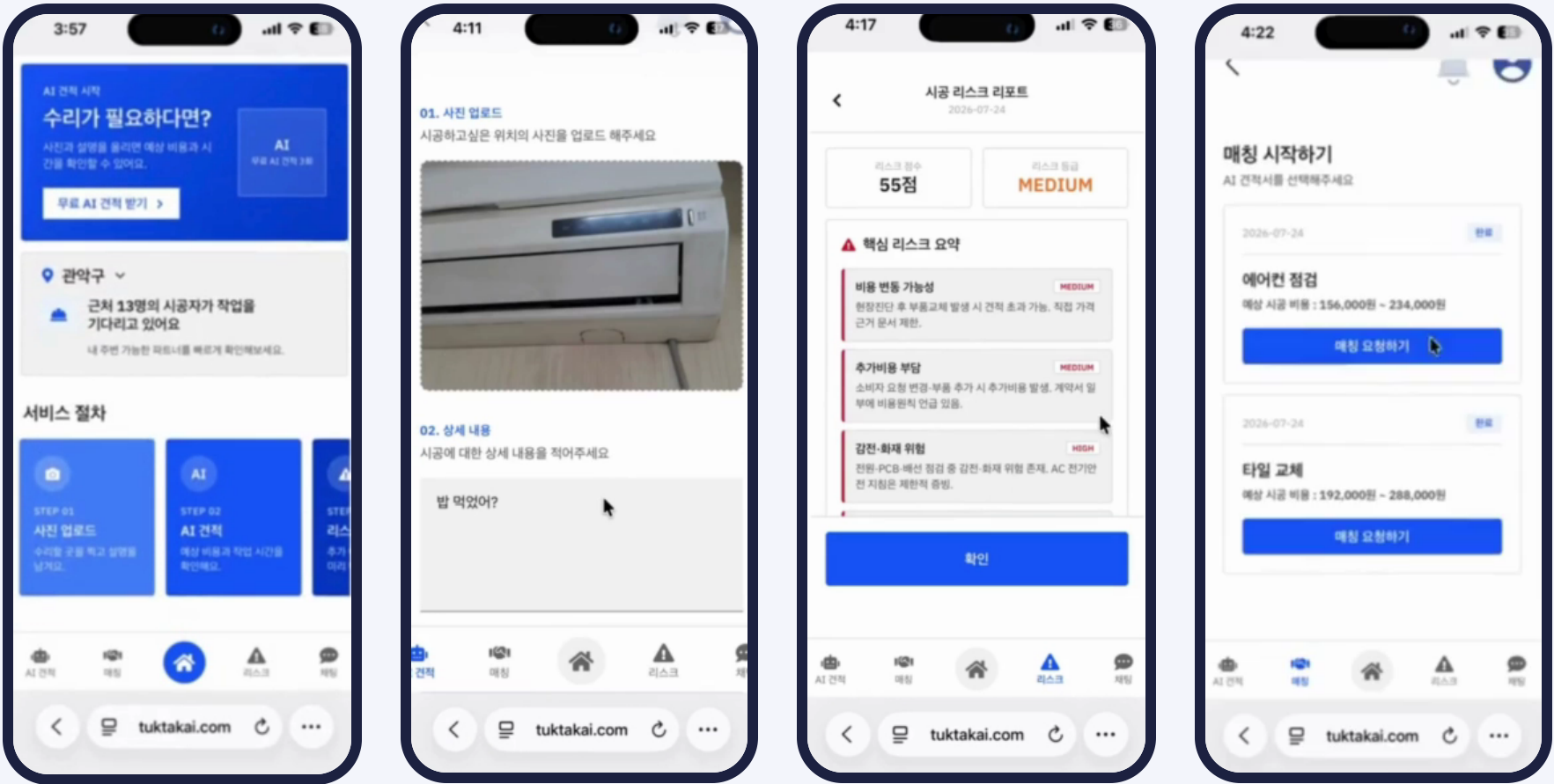
OpenAI API

React

Docker

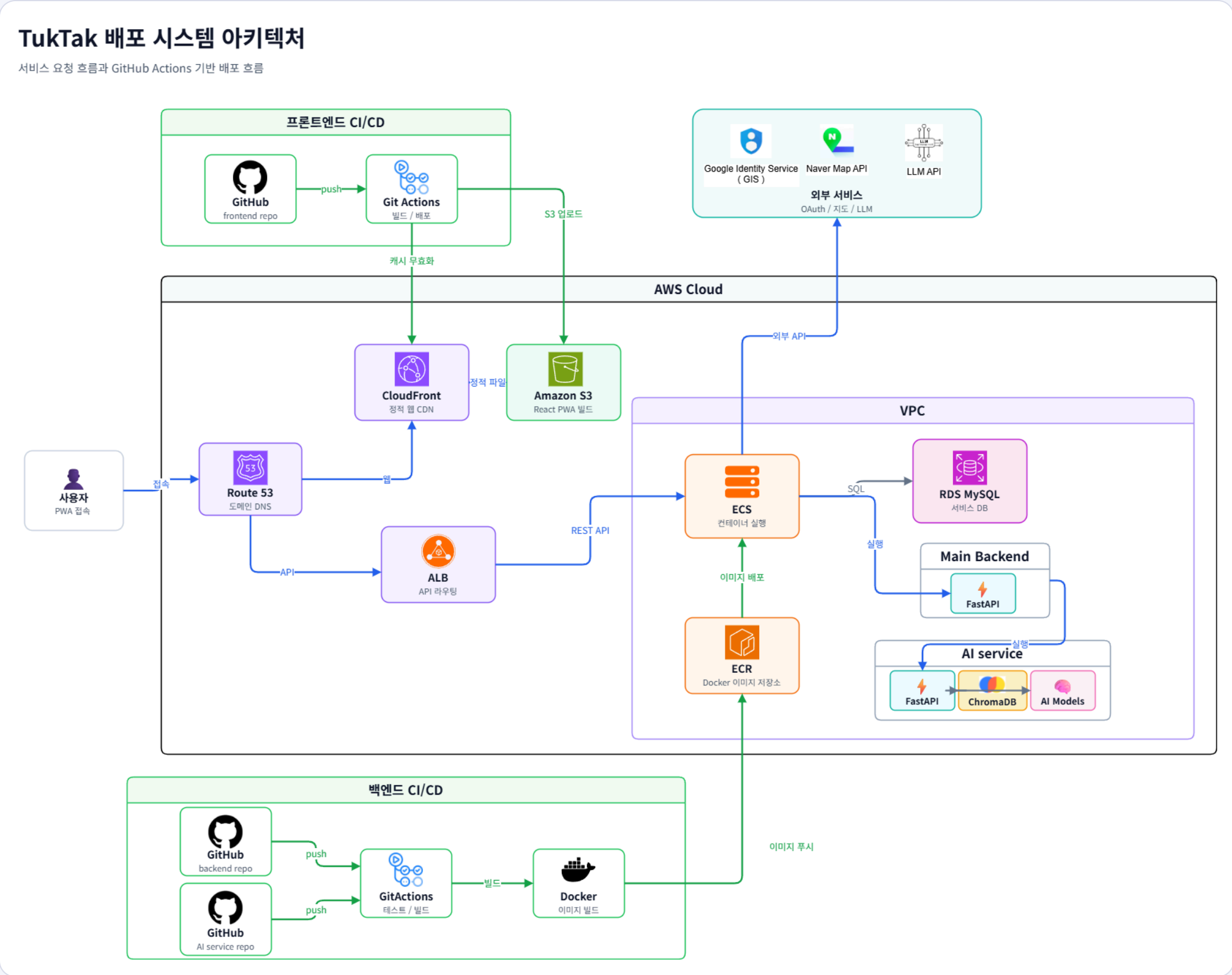
AWS ECS · S3 · CloudFront

GitHub Actions



홈 → 사진 업로드 → AI 리스크 리포트 → 시공사 매칭 (시연 영상 캡처)

React · Main Backend · AI Service를 분리한 구조



S3 · CloudFront(정적 웹) / ALB → ECS(Main Backend, AI Service) / RDS MySQL / ChromaDB · GitHub Actions로 Docker 이미지 빌드 · ECR 푸시

담당 업무

- RAG 리스크 문서 검색 설계 · 구현 — 근거 문서 71개 임베딩, 리스크 항목 5개별 메타데이터 필터
- Risk Report 도메인 모델 · API — 생성 · 저장 · 조회, Main Backend ↔ AI Service 연동
- 이미지 유사사례 검색 — 임베딩 모델 4종 비교, 카테고리별 threshold 설계
- 수리 요청 구조화 모델 검증 — KLUE-RoBERTa vs KoELECTRA vs LLM 후보
- React 화면 — 고객 · 시공사 마이페이지, 홈, AI 리스크 페이지 연동
- 배포 — AI Service Dockerfile, GitHub Actions 빌드 워크플로우

근거가 부족하면 FAILED로 분리해 잘못된 위험도 노출을 막았습니다

유사도 점수 하나로 결과를 확정하던 구조를 리스크 항목 5개로 나누고, UNKNOWN이 3개 이상이면 FAILED로 처리했습니다. 10개 카테고리 중 근거가 부족한 1개가 분리됐습니다.

정확도 대신 오답 통과율로 팀 기준을 바꿨습니다

잘못된 사례가 견적 근거로 쓰이는 비용이 더 크다고 보고, 오답 통과 0개를 1순위로 두었습니다. 그 기준에서 정답 통과율 60.4%를 감수했습니다.

입력 검증 → 구조화 → 유사사례 → 견적 → RAG 리스크 리포트



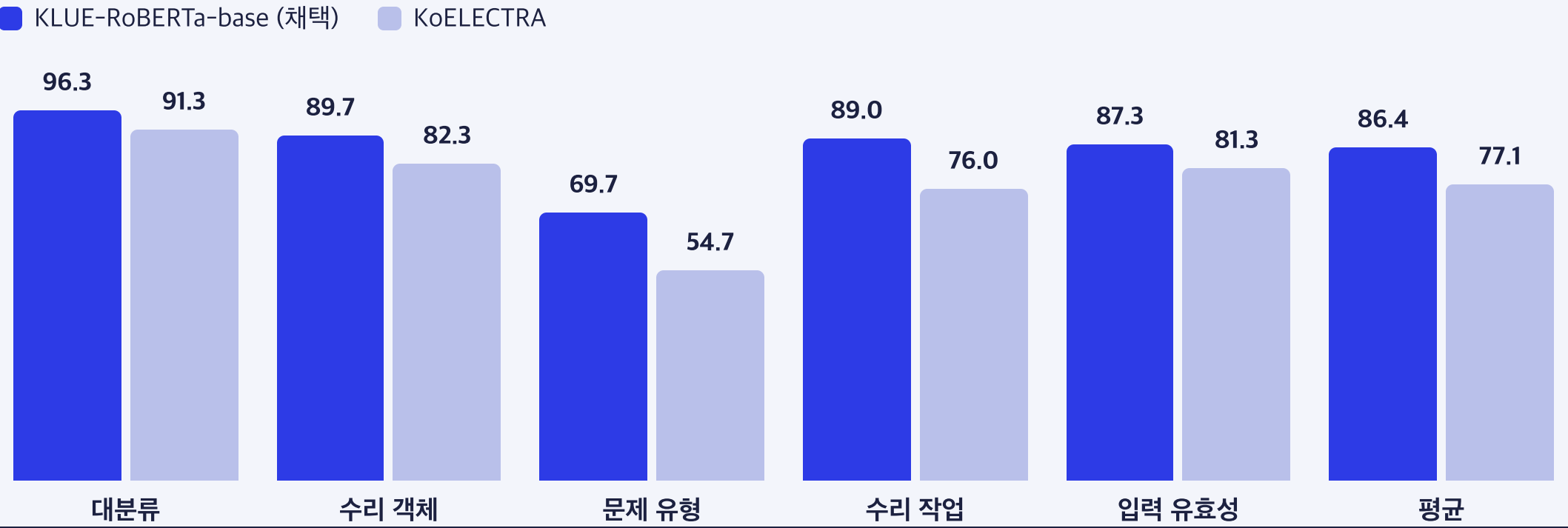
RAG 리스크 항목별 threshold · 가중치 (직접 설계)

리스크 항목	threshold	가중치	의미
PRICE	0.30	10%	가격 · 견적 리스크 (통계 자료는 가격 근거에서 제외)
EXTRA_COST	0.55	20%	추가 공사 · 추가 비용
SAFETY	0.60	30%	작업 중 안전 · 사용자 안전
CONTRACT	0.60	25%	계약 · 하자보수 · AS · 분쟁
FIELD	0.65	15%	현장 방문 전 확인하기 어려운 변수

설계 판단

- 임베딩 모델 — BGE-M3(1024차원)보다 가볍고 운영 부담이 적은 Ko-SRoBERTa(768차원) 채택. 한국어 요약문 · 메타데이터 중심 문서에 안정적
- 실패 기준 — 항목 5개 중 UNKNOWN 3개 이상이면 리포트를 FAILED로 분리. 10개 카테고리 테스트에서 9개 COMPLETED, 1개 FAILED
- 가격 근거 — 소비자 피해구제 통계(STAT)는 특정 수리의 직접 가격 근거가 아니므로 PRICE 검색에서 제외
- 구조화 모델 — LLM 후보 3종 100건 평가에서 Qwen 오류 78건 · 지연 27.7초. 구조화는 소형 모델 (KLUE-RoBERTa, 0.014초)로 결정

구조화 모델 선정 — 5개 필드 정확도 비교



4차 최종 서비스 적합성 평가 · Accuracy(%) · missing_info 제외 5개 구조화 필드

86.4%

평균 정확도 (KoELECTRA 77.1%)

0.014초

요청당 처리시간 · LLM 호출 없음

4단계

기본 수행 → 구조화 정확도 → 예외 케이스 → 서비스 적합성

평가 설계

- 1차 기본 수행능력: 출력 성공 · 형식 준수 · 라벨 범주 준수 · 처리시간으로 KoT5 · EXAONE 제외
- 2차 구조화 정확도: 필드별 Accuracy · Macro-F1 · Exact Match. Qwen2.5-Instruct 제외
- 3차 예외 케이스: 모호한 입력 · 위험 요청 · 정보 부족 입력의 validity · missing_info 판정
- 4차 서비스 적합성: 5개 필드 정확도와 처리시간으로 최종 선정

이미지 유사사례 검색

- 후보: CLIP · DINOv2 · ConvNeXt · Nomic Embed Vision을 같은 기준으로 비교
- 카테고리마다 시각 난이도가 달라 공통 threshold 대신 카테고리별 threshold 적용
- 선정 우선순위: ① 오답 통과 최소화 ② 정답 통과 확보 ③ 부족 시 LLM · 기준으로 분기
- 결과: 오답 통과 0개, 정답 통과율 60.4% (전체 1위)

문제 해결 과정

근거 부족 케이스가 낮은 위험도로 정상 출력되던 문제

상황

근거 문서가 5개 중 2개뿐인 카테고리(가전)가 낮은 위험도로 정상 출력되고, 통계 자료가 가격 근거로 통과했습니다.

원인

유사도 점수만으로 결과를 확정하고, 리스크 항목별 근거 수와 문서 유형을 확인하지 않았습니다.

행동

검색을 리스크 항목 5개로 나누고 문서 유형 · 카테고리 메타데이터 필터를 적용했습니다. UNKNOWN 3개 이상이면 FAILED로 처리하는 기준을 세웠습니다.

결과

근거 부족 카테고리가 FAILED로 분리되어 잘못된 위험도 노출을 차단했습니다. 좋은 결과보다 실패 조건을 먼저 정의하는 기준을 얻었습니다.

이미지 검색 기준에 대한 팀 의견 차이

상황

유사사례를 최대한 많이 보여주자는 의견과 틀린 사례가 하나라도 통과 되는 것을 막자는 의견이 갈렸습니다.

원인

정확도라는 단일 지표로만 평가해, 잘못된 사례가 견적 근거로 쓰일 위험과 카테고리별 난이도 차이가 드러나지 않았습니다.

행동

평가 지표를 오답 통과율로 바꿔 다시 측정하자고 제안했습니다. 카테고리별 보수적 threshold를 적용하고, 애매한 경우는 LLM 판단으로 분기했습니다.

결과

오답 통과 0개 · 정답 통과율 60.4%라는 수치를 함께 보고 팀이 합의했습니다. 의견 차이는 설득이 아니라 같은 기준의 측정으로 좁혀진다는 것을 배웠습니다.

한국어 리스크 문서 검색 품질 저하

상황

LangChain 기본 임베딩 모델로는 한국어 근거 문서 검색 결과가 부정확했습니다.

원인

영문 위주 임베딩 모델이 한국어 요약문과 메타데이터 중심 문서에 맞지 않았습니다.

행동

임베딩 모델 성능평가를 직접 수행해 Ko-SRoBERTa로 교체했습니다. BGE-M3와 차원이 달라 ChromaDB 컬렉션을 분리했습니다.

결과

한국어 근거 문서 검색 품질이 개선됐고, 리스크 유형별 Top-K · threshold 기준을 확정했습니다.

Investment Agent Report

RAG + LangGraph Agent 기반 투자 리포트 데모

기업명 · 티커를 입력하면 뉴스와 재무 데이터를 수집해 RAG 검색과 Agent 분석으로 참고 리포트를 생성합니다. 부트캠프 수료 다음 날 시작해 6일 만에 수집부터 API까지 혼자 만들었습니다.

기간	2026.08.05 ~ 2026.08.10 (6일)
인원	1명 · 전체 설계 · 구현
구조	Rule-based baseline으로 만들고 LLM Agent로 교체 가능한 구조

주요 기능

- Google News RSS 뉴스 수집, yfinance 재무 데이터 수집과 실패 처리
- Sentence-Transformers 임베딩 + ChromaDB 벡터 검색
- Planner · RAG · News · Finance · Risk · Report 6개 Agent 워크플로우
- FastAPI 리포트 생성 · 저장 · 조회 API 4개, Streamlit 데모, Docker Compose

기술 스택

Python 3.11

FastAPI

SQLAlchemy

LangGraph

ChromaDB

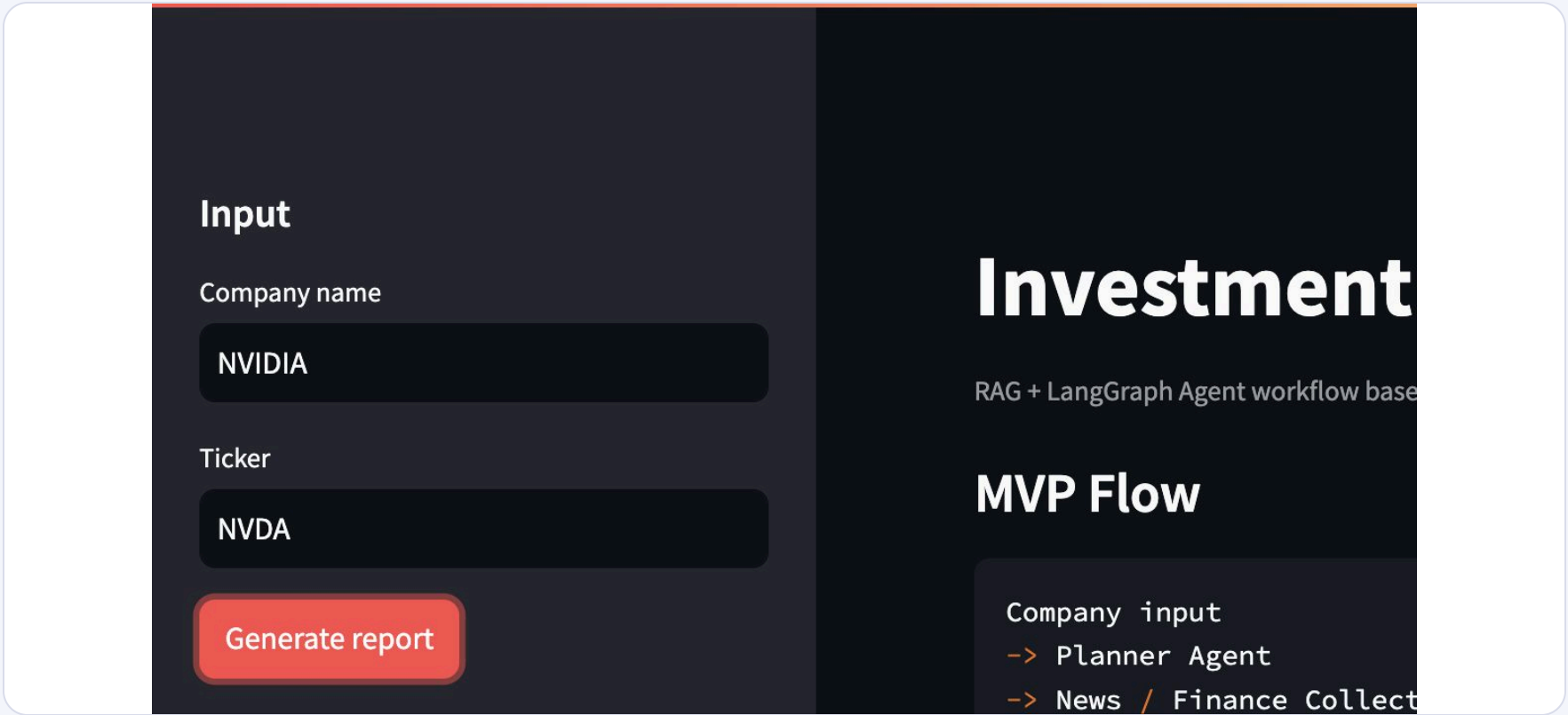
Sentence-Transformers

Streamlit

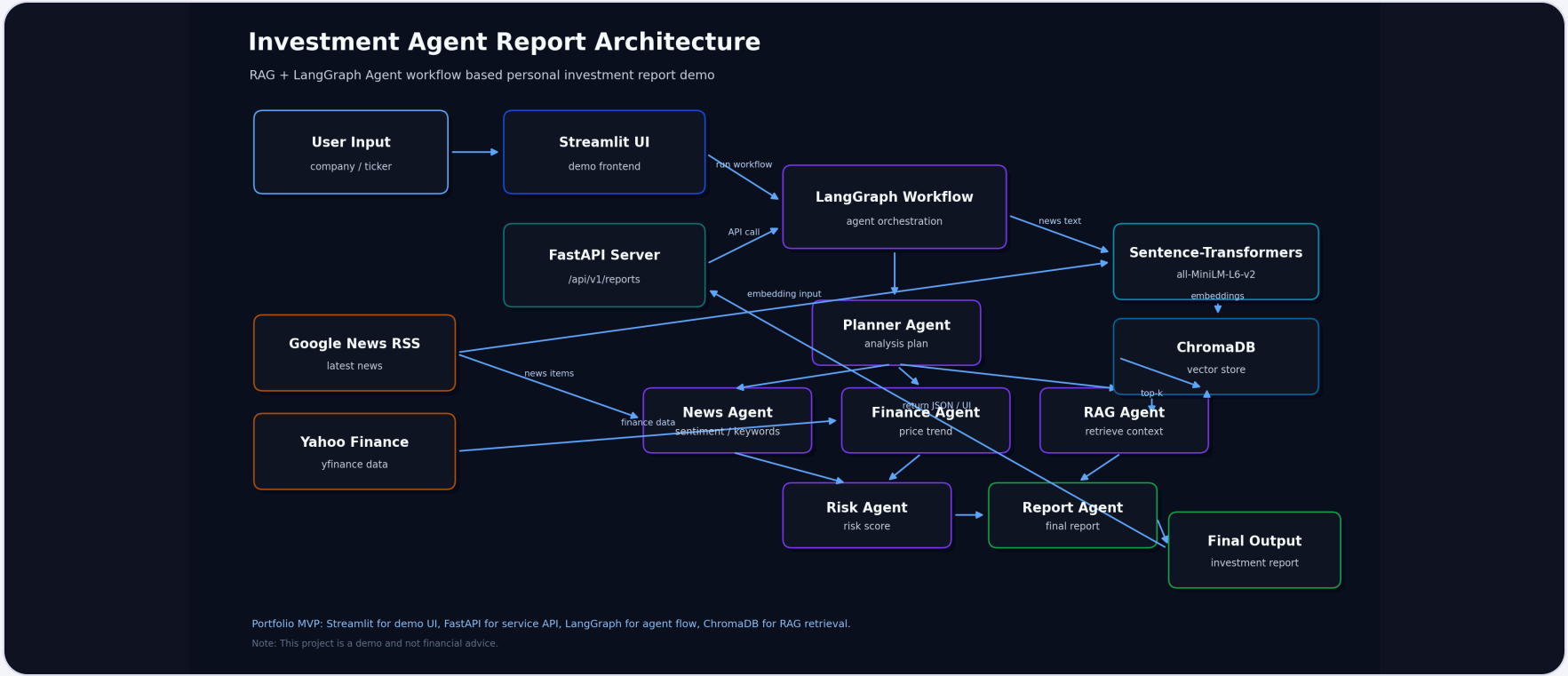
yfinance

Docker Compose

GitHub · github.com/JW020799/investment-agent-report



Streamlit 데모 화면 — 기업명 · 티커 입력 후 리포트 생성



아키텍처 — Streamlit / FastAPI → LangGraph 워크플로우 → Agent 6개 → ChromaDB (README 수록)

Agent 6개 워크플로우와 실패 처리



외부 API가 실패해도 리포트 생성을 멈추지 않게 했습니다

yfinance 429(Too Many Requests)로 전체 생성이 중단되던 문제를 fast_info · 최근 가격 이력 우선 사용으로 줄이고, 실패 시 status: unavailable로 반환해 뉴스 · RAG · Agent 분석만으로 리포트를 유지했습니다. 실패 원인은 응답에 그대로 남깁니다.

근거가 약한 리포트를 사용자가 바로 알 수 있게 했습니다

경량 임베딩과 소량의 최신 뉴스만 대상으로 검색해 유사도 분포가 낮은 한계를 숨기지 않고, 유사도 점수와 참조 문서를 UI · API 응답에 그대로 노출했습니다. 임베딩 모델 비교 · threshold 필터링 · 뉴스 중복 제거는 확장 과제로 문서화했습니다.

API

Method	Endpoint	설명
POST	/api/v1/reports	리포트 생성 (워크플로우 실행)
GET	/api/v1/reports	리포트 목록
GET	/api/v1/reports/{id}	리포트 단건 조회
GET	/health	상태 확인

Docker Compose로 API · UI 두 서비스 동시 실행 · README에 아키텍처 · API 명세 · 트러블슈팅 문서화

PROJECT 03 · TEAM

GIUT

공공데이터와 AI를 활용한 지역 행사 탐색 · 추천 코스 서비스

한국관광공사 공공데이터로 지역 행사를 탐색하고, OpenAI API로 지역 · 날짜 · 취향에 맞는 행사 코스를 추천하는 웹 서비스입니다.

기간	2026.04.13 ~ 2026.05.27 (7주)
인원	5명 · IBM x Red Hat AI 부트캠프 중간 프로젝트
역할	백엔드 모델 · API, 화면 일부

담당 업무

- Event · EventDetail · Course 모델과 Pydantic 스키마 설계, Course API 구현
- JWT + HttpOnly Cookie 기반 회원가입 · 로그인 API
- 지역 · 날짜 · 키워드 조건 행사 검색 · 필터링 API
- GPT 추천 코스 저장 · 마이페이지 조회 API, React 주요 화면 연동
- AWS EC2에 Docker Compose로 배포

기술 스택

FastAPI

MySQL

SQLAlchemy

JWT

React

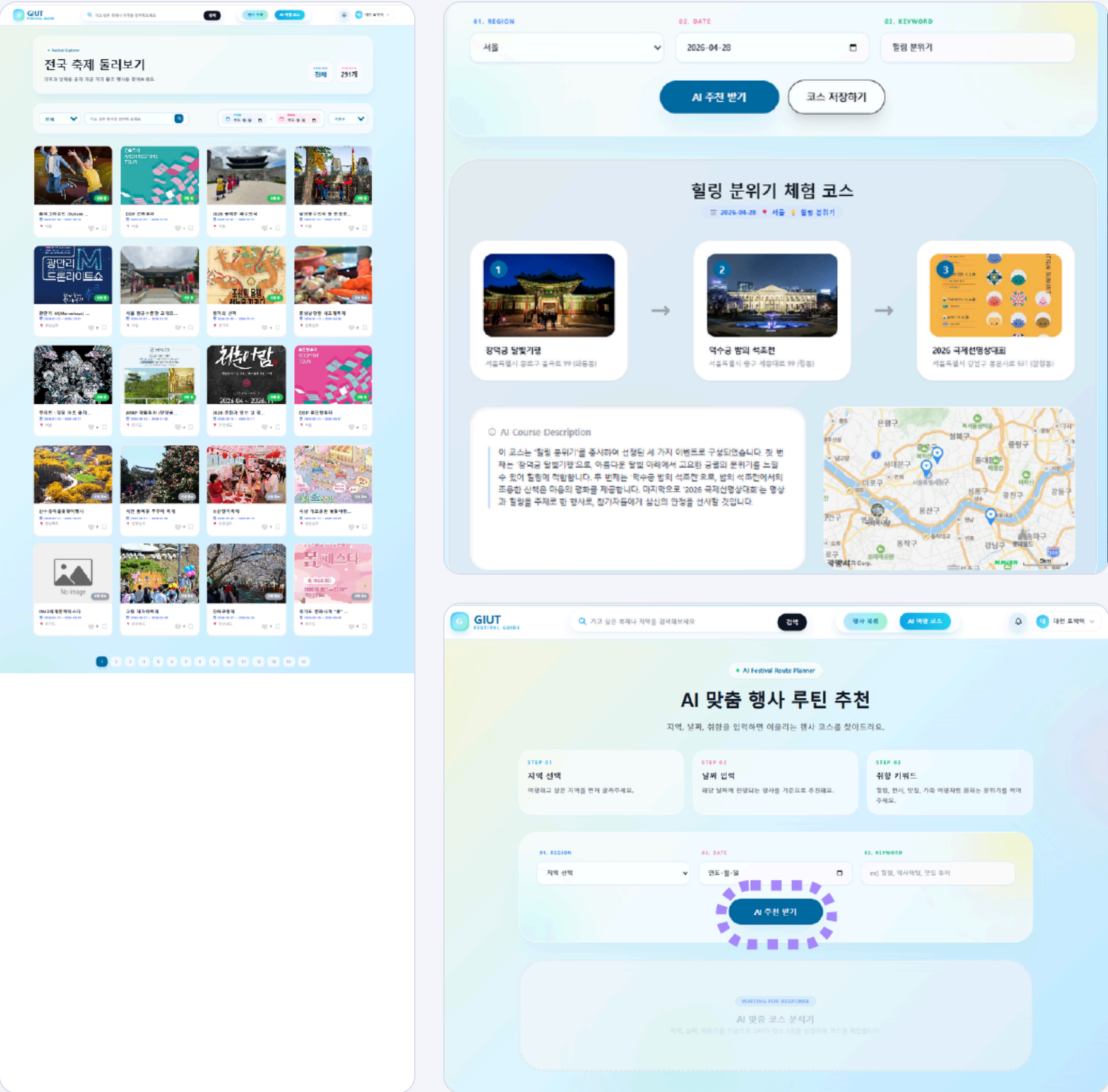
OpenAI API

한국관광공사 API

Naver Map API

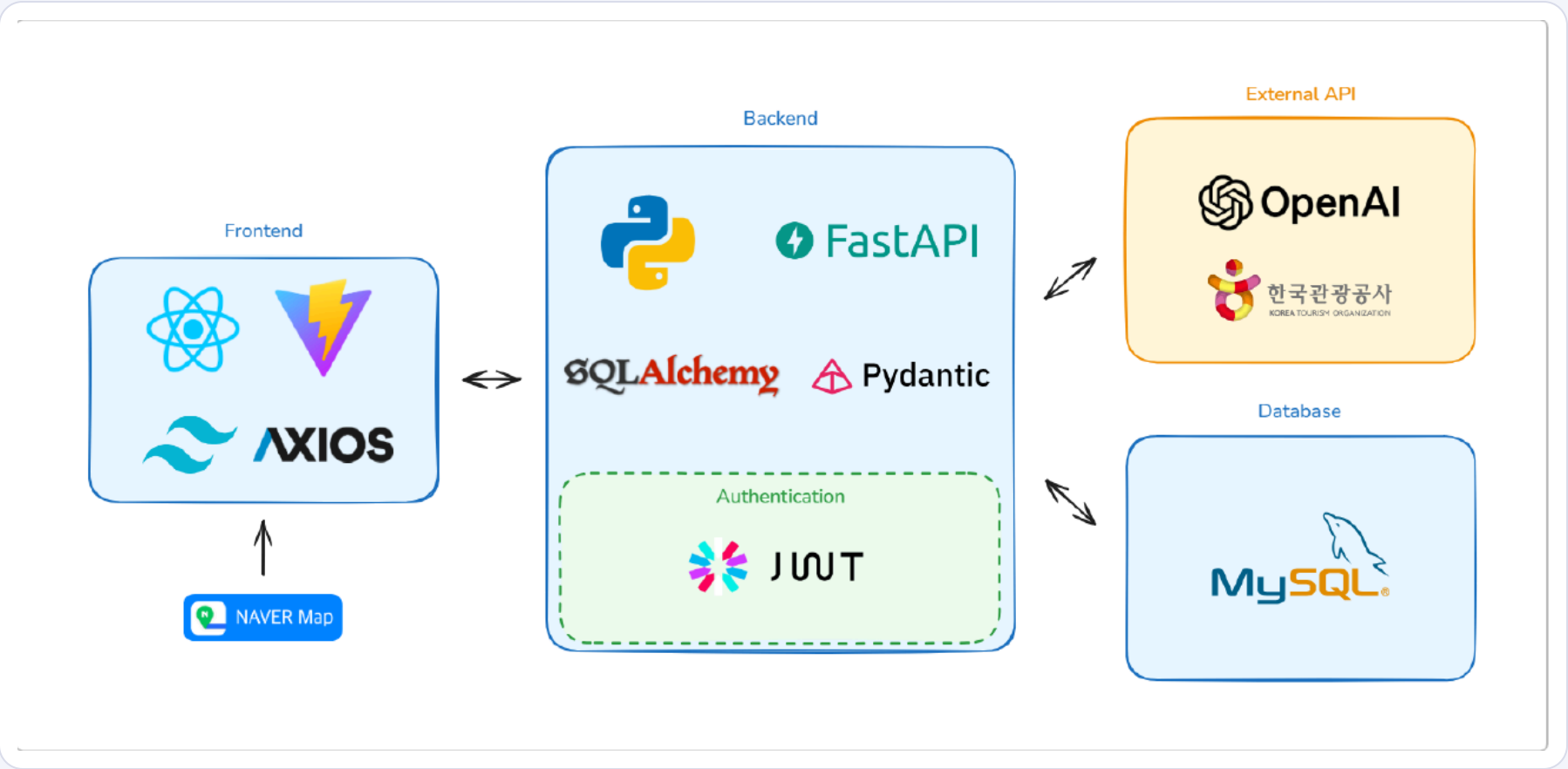
Docker Compose

AWS EC2



행사 목록(지역 · 날짜 · 검색 필터) / AI 행사 코스 추천 입력 · 결과

외부 API 구조가 아니라 서비스 요구에서 모델을 시작했습니다



React · Vite · Axios ↔ FastAPI · SQLAlchemy · Pydantic · JWT ↔ MySQL / OpenAI · 한국관광공사 API / Naver Map

Event 모델 재설계 — 컬럼 19개 → 10개

구분	재설계 전	재설계 후
기준	한국관광공사 API 응답 필드를 그대로 옮김	화면에 필요한 필드 중심
컬럼	19개 (contentid, mapx/mapy, lclsSystem3 ...)	10개 (제목 · 지역 · 기간 · 장소 · 이미지 · 설명 · 좋아요 수 ...)
이름	모든 컬럼에 event_ 접두어, 스키마와 불일치	접두어 제거, 모델 · 스키마 필드명 일치
범위	Event만	EventDetail · Course도 같은 기준으로 정리

화면에서 쓰지 않는 필드까지 관리하던 비용이 사라졌습니다

외부 API 응답 구조를 그대로 옮기면 처음엔 빠르지만, 스키마 불일치와 불필요한 필드 관리가 뒤 따릅니다. 데이터 모델은 서비스 요구에서 시작해야 한다는 기준을 얻었습니다.

5명 팀에서 Branch · PR 기반으로 협업했습니다

회원가입부터 조건 검색까지 API 전 구간을 맡아 구현하고, 공공데이터 · OpenAI · MySQL을 연 동한 서비스를 EC2에 배포했습니다.

교육과 지금 하고 있는 것

2026.02 ~ 2026.08 · 수료

IBM x Red Hat AI 부트캠프

Python · FastAPI · React · MySQL · Docker · AWS 기반
웹 서비스 개발 과정. 중간 프로젝트 GIUT, 최종 프로젝트
TUKTAK 수행.

2026.08.23 ~ 2026.10.22 · 진행 중

심화 생성형 AI 활용 인재양성 부트캠프

기성 모델을 가져다 쓰는 단계를 넘어, 직접 학습시킨 모델
을 서비스에 연결하는 방법을 익히는 중입니다.

2018.03 ~ 2021.02

경민대학교

호텔관광경영과 졸업

다음 프로젝트에서는 직접 학습시킨 모델을 메인으로 둡니다

지금까지는 기성 모델을 비교 · 검증해 서비스 기준에 맞게 연결하는 일을 했습니다. 진행 중인 파이널 프로젝트에서는 데이터 구축 → 모델 비교 → 파인튜닝 → 전 · 후 수치 비교까지 직접 학습한 모델로 남기는 것을 목표로 하고 있습니다.

THANK YOU

감사합니다.

AI 결과를 검증해서 서비스에 안전하게 연결하는 일을
계속 잘하고 싶습니다.

전지원

020799@naver.com · github.com/JW020799 · jw020799.github.io